

VIBEWEATHER

Full-Stack Weather Intelligence Web Application

English Edition • Bilingual Korean-English Toggle Edition

CASE STUDY & TECHNICAL DOCUMENTATION

Developed by

SweetyCodes

Computer Science Student · Frontend Developer · Python Automation

TECH STACK

HTML5 • CSS3 Glassmorphism • JavaScript ES6+ • WeatherAPI.com • PHP 8.x

GitHub Pages • localStorage • Web Share API • Nominatim Geocoding

Table of Contents

Chapter 1 Project Genesis & Vision

- 1.1 The Spark — A Real Conversation
- 1.2 The Five Pillars
- 1.3 Design Philosophy
- 1.4 What Makes VibeWeather Different

Chapter 2 Product Overview & Feature Set

- 2.1 High-Level Feature Map
- 2.2 The 12 Lifestyle Intelligence Cards
- 2.3 Weather Data Sources
- 2.4 Key Metrics at a Glance

Chapter 3 Version History — Two Editions

- 3.1 Version 1: English Edition
- 3.2 Version 2: Bilingual KO/EN Toggle Edition
- 3.3 Version Comparison Table
- 3.4 Why Build Two Versions?

Chapter 4 Screenshots & UI Walkthrough

- 4.1 Portal / Landing Page
- 4.2 Main Weather Card
- 4.3 Lifestyle Vibe Grid
- 4.4 Solar Events & Moon Phase
- 4.5 5-Day & Hourly Forecast
- 4.6 Mobile Experience (KO Mode)
- 4.7 Dynamic Backgrounds

Chapter 5 Architecture & System Design

- 5.1 Three-Tier Architecture
- 5.2 Data Flow Diagram
- 5.3 File Structure
- 5.4 API Integration
- 5.5 Proxy Pattern (Version 1)

Chapter 6 Core Algorithms & Science

- 6.1 Real Feel — 3-Branch Temperature Model
- 6.2 Dew Point & Humidity Comfort Scale
- 6.3 Wind Feeling Index
- 6.4 UV, AQI, Visibility & Pressure Scales
- 6.5 Outfit Recommendation Logic
- 6.6 "Coming Up" Predictive Widget
- 6.7 Alert Generation System

Chapter 7 Bilingual System — KO/EN Engine

- 7.1 Language Toggle Architecture
- 7.2 The t() Helper Function
- 7.3 Korean City Coordinate System
- 7.4 Nominatim Geocoding Fallback
- 7.5 Bilingual Date & Time Formatting
- 7.6 Re-render on Language Switch

Chapter 8 Frontend Engineering

- 8.1 Glassmorphism Design System
- 8.2 CSS Animation Library (6 Motion Classes)
- 8.3 Dynamic Video Background Engine
- 8.4 Ambient Audio Engine
- 8.5 Responsive Grid System
- 8.6 Live Clock & Timezone Handling

Chapter 9 Problems & Solutions

- 9.1 Korean City API Problem
- 9.2 Language Toggle & Computed Strings
- 9.3 Wrong Timezone Background
- 9.4 Real Feel Inaccuracy
- 9.5 Audio Autoplay Block
- 9.6 Mobile Grid Collapse
- 9.7 API Key Exposure

Chapter 10 Testing & Quality Assurance

- 10.1 Test Strategy
- 10.2 Unit Test Cases
- 10.3 Cross-Browser Results
- 10.4 Performance Metrics
- 10.5 Mobile Testing

Chapter 11 Feature Comparison & Market Context

- 11.1 VibeWeather vs. Existing Apps
- 11.2 The "Vibe" Differentiator
- 11.3 Target User Personas

Chapter 12 Key Learnings & Reflections

- 12.1 Technical Learnings
- 12.2 Design Learnings
- 12.3 Personal Reflections

Chapter 13 Future Roadmap

- 13.1 Short-Term (0-3 months)
- 13.2 Medium-Term (3-12 months)
- 13.3 Long-Term Vision

Appendix Reference Tables & Code

- A WeatherAPI JSON Field Map
- B Condition Code to Icon Map
- C Key JavaScript Functions Reference
- D Deployment Checklist

Project Genesis & Vision

1.1 The Spark -- A Real Conversation

VibeWeather was not born from a brief or a curriculum requirement. It started with a real question asked to someone who matters:

“What would your ideal weather app look like?”

The answer was immediate and specific: "I want to know what it actually feels like outside before I get dressed. Not just 28 degrees -- does it feel like 28? Is the wind making it worse? What should I wear? And I want it to look alive, not like a spreadsheet with clouds."

That five-second response became the entire product brief. Every feature, every design decision, every line of code in VibeWeather traces back to those words. This is the rare kind of clarity that most products spend months of user research trying to find -- and it arrived in a conversation.

1.2 The Five Pillars

From that conversation, five non-negotiable requirements emerged. These are not features in the traditional sense -- they are design pillars. Everything else in the app exists to serve them:

01 Real Feel Temperature

Not the raw number from the API. A scientifically computed apparent temperature that accounts for wind chill, heat index, and humidity -- using three separate meteorological models. When you step outside, this is what your skin actually registers.

02 Outfit Recommendations

A priority-ordered decision tree that reads the weather and tells you what to wear. Thunder? Stay inside. Rain? Umbrella. Snow? Bundle up. Below 12 degrees? Parka. Above 28? Tank top. Each outfit suggestion gets an animated emoji that moves with the weather mood.

03 Lively, Immersive Design

Dynamic video backgrounds that change with the time of day in the searched city -- not the user's device time. Morning, afternoon, evening, night -- each has its own video atmosphere. Glassmorphism cards float over the living background.

04 Ambient Audio

Rain sounds when it rains. Sunny upbeat audio on clear days. Cozy night ambience after dark. Winter sounds during snow. The app does not just show the weather -- it puts you in it. Sound is user-controlled via a mute toggle.

05 Bilingual KO/EN Support

Every element -- greetings, card labels, comfort descriptions, day names, forecast text, outfit advice -- switches between Korean and English in real time with a single tap. No reload. No data re-fetch. Sub-50ms toggle.

1.3 Design Philosophy

VibeWeather was conceived with a deliberate philosophy that runs counter to most modern web development practice:

- **Zero framework overhead.** No React, no Vue, no Angular. Pure vanilla JavaScript. This was a conscious choice -- without a framework's state system, every piece of data flow had to be designed manually. The result is a deeper understanding of what frameworks actually solve.
- **Science over approximation.** Every computed value uses real meteorological models. The real feel temperature uses three separate scientific formulae. Humidity comfort uses dew point rather than raw percentage. This commitment to accuracy is what separates VibeWeather from apps that just relay API numbers.
- **Emotion through atmosphere.** A weather app can tell you it's 18 degrees and raining. VibeWeather shows you the grey sky through a living video, wraps your data in a frosted glass card, and plays rain sounds softly in the background. Information and feeling together.
- **Bilingual from the ground up.** The Korean language support was not added as an afterthought -- it required redesigning how every string in the application is managed. This architectural choice made the codebase fundamentally more maintainable and extensible.

1.4 What Makes VibeWeather Different

Most weather apps present data. VibeWeather presents an experience. The difference is not cosmetic -- it reflects a fundamentally different question being asked:

Typical Weather App asks:	VibeWeather asks:
"What is the temperature?"	"What will it feel like on your skin?"
"What is the humidity?"	"Is the air dry, sticky, or oppressive?"
"What is the weather condition?"	"What should you wear? Can you go for a run?"
"What time is sunrise?"	"Is it morning, afternoon, or night there right now?"
"What does the 5-day forecast say?"	"What will change in the next 5 hours?"

Product Overview & Feature Set

2.1 High-Level Feature Map

VibeWeather delivers data across five distinct categories of information, each rendered through a distinct UI component:

Category	Features	UI Component
Current Conditions	Temperature, Real Feel, Condition, Description, Wind Speed	Main Card
Precipitation	Daily chance, animated progress bar, hourly chance per card	Main Card + Hourly Strip
Daily Range	Min/max temperature, animated gradient marker	Main Card
Lifestyle Intelligence	12 cards: Outfit, UV, AQI, Humidity, Wind, Pressure, Visibility, Run Guide, Pollen, Real Feel, Coming Up	Main Card
Solar & Lunar	Sunrise, Sunset (animated arcs), Moon phase with emoji	Extra Container
Forecast	5-day daily + 12-hour hourly with icons and rain %	Forecast Strips
Atmosphere	Dynamic video background (4 time-of-day videos)	Full-page video layer
Audio	Ambient soundscape matching weather condition	Audio Engine
Alerts	Thunder, blizzard, high UV, hail, extreme heat/cold banners	Alert Banner
Sharing	Native Web Share API + clipboard fallback	Share Button
History	Last searched city with temperature chip	Portal Page

2.2 The 12 Lifestyle Intelligence Cards

The Vibe Grid is the heart of what differentiates VibeWeather from a standard weather display. Each card answers a specific human question:

01. Outfit

User question answered: "What should I wear today?"

Priority-ordered decision tree evaluating condition code + temperature. Returns emoji + label + CSS animation class.

02. Real Feel

User question answered: "Will it feel hotter or colder than the thermometer says?"

3-branch model: Wind Chill (cold+windy), Rothfusz Heat Index (hot+humid), Steadman Apparent Temp (moderate).

03. Humidity (Dew Point)

User question answered: "Will the air feel sticky or comfortable?"

Computes dew point via Magnus approximation, maps to 9-level comfort scale from "Very Dry" to "Soupy/Unbearable".

04. UV Index

User question answered: "Do I need sunscreen?"

6-level scale from None to Extreme. Bilingual label. Color coded from white to deep red.

05. Air Quality (AQI)

User question answered: "Is it safe to breathe outside?"

US EPA 6-level index (Good/Moderate/USG/Unhealthy/Very Unhealthy/Hazardous) from API air_quality data.

06. Visibility

User question answered: "How far can I see?"

5-level label from Crystal Clear (10+ km) to Very Poor (<1 km). Includes color indicator.

07. Pressure

User question answered: "Is a storm coming?"

6-level scale from Very Low/Stormy to Very High/Dry. Maps hPa reading to atmospheric stability.

08. Wind Speed

User question answered: "Is it windy enough to notice?"

Raw wind_kph value paired with Real Feel status message for wind-chill context.

09. Running

User question answered: "Is today good for a run?"

Composite rule: temp 12-25 degC AND no rain/drizzle AND no fog AND rain chance below 50%.

10. Driving

User question answered: "Is it safe to drive?"

Rule: wind over 20 km/h OR any rain OR fog triggers Caution. Otherwise Smooth driving confirmed.

11. Pollen

User question answered: "Will my allergies act up?"

Sunny clear condition (code 1000) during daytime returns High pollen. All others return Low.

12. Coming Up

User question answered: "What is about to change?"

5-hour lookahead into hourly forecast data. Predicts imminent rain, warming, cooling, or wind increase.

2.3 Weather Data Sources

All weather data comes from WeatherAPI.com, a RESTful JSON API. The single endpoint call for each city search returns all required data in one request, minimizing latency:

API Field Path	Data Returned	Used For
current.temp_c	Current temperature in Celsius	Main card, real feel input
current.feelslike_c	API apparent temp (overridden)	Not used -- replaced by custom formula
current.humidity	Relative humidity %	Dew point calc, humidity card

current.wind_kph	Wind speed in km/h	Wind card, real feel input, driving card
current.uv	UV index (0-11+)	UV card label and color
current.vis_km	Visibility in km	Visibility card
current.pressure_mb	Atmospheric pressure in hPa	Pressure card
current.air_quality.us-epa-index	EPA AQI level 1-6	Air quality card
current.condition.code	Numeric weather condition code	Icon selection, outfit logic, alerts
current.is_day	0 or 1	Day/night icon selection, pollen estimate
location.tz_id	IANA timezone string	Live clock, background video selection
location.localtime	City local time string	Time-of-day calculation
forecast.forecastday[0].astro	Sunrise, sunset, moon phase	Solar row, moon card
forecast.forecastday[0].day	Daily min/max temp, rain chance	Temperature bar, precip bar
forecast.forecastday[0].hour[]	Hourly conditions array	Hourly strip, Coming Up widget
forecast.forecastday[1-5]	Next 5 days conditions	5-day forecast strip

2.4 Key Metrics at a Glance

35+ DOM elements updated per search	12 Lifestyle intelligence cards	150+ Korean cities in coordinate map	3 Scientific temperature models
4 Video background assets	4 Ambient audio tracks	9 CSS keyframe animations	6 Weather condition flag types

Version History -- Two Editions

3.1 Version 1: English Edition

The English edition was the foundation -- built to prove the concept and establish the architecture. It introduced every core feature of VibeWeather: the glassmorphism design, the real feel computation, the 12 lifestyle cards, the dynamic video background, and the hourly/daily forecast system.

Version 1 uses a PHP backend proxy to protect the API key server-side. This is a standard security pattern for client-side applications that require API keys -- the browser never sees the key, only the JSON response. The PHP layer also provides a date formatting microservice (getdate.php) that returns locale-aware date strings for any IANA timezone.

Version 1 Technical Specification

Frontend: HTML5, CSS3, JavaScript ES6+ (vanilla, no frameworks)

Backend: PHP 8.x -- API proxy (weather.php) + date service (getdate.php)

API: WeatherAPI.com /v1/forecast.json with AQI enabled

Deployment: Apache/Nginx + PHP server (XAMPP local, cPanel production)

Storage: localStorage for last searched city persistence

Language: English only

3.2 Version 2: Bilingual KO/EN Toggle Edition

Version 2 is the production edition currently deployed on GitHub Pages. It builds on every feature of Version 1 and adds three major new systems:

- **Real-time bilingual toggle.** A KO | EN button in the top-left corner switches the entire UI between Korean and English without any reload or API re-fetch. Every text element -- including dynamically computed comfort labels, greeting messages, day-of-week names, and outfit advice -- switches in under 50ms.
- **Korean city support.** A 150+ entry hardcoded dictionary maps Korean city and district names to GPS coordinates. WeatherAPI does not understand Korean text, so all Korean input is converted to lat/lon before the API call. Cities not in the dictionary fall back to Nominatim geocoding.
- **Active ambient audio engine.** Four looping audio tracks are selected based on weather condition: rain.mp3, winter.mp3, sunny.mp3, night.mp3. Version 1 had the audio element reserved but inactive; Version 2 activates the full engine with user-controlled mute toggle.

Version 2 Technical Specification

Frontend: HTML5, CSS3, JavaScript ES6+ (vanilla, no frameworks)

Backend: None -- fully static, API called directly from client

API: WeatherAPI.com /v1/forecast.json with AQI enabled (key in JS)

Deployment: GitHub Pages (static hosting, no server required)

Storage: localStorage for last city + language preference

Languages: Korean (KO) and English (EN) with real-time toggle

Korean support: 150+ city coordinate dictionary + Nominatim geocoding fallback

Audio: 4-track ambient audio engine (rain, winter, sunny, night)

Live demo: sweety-hjh223.github.io

3.3 Version Comparison Table

Feature / Aspect	Version 1 (English)	Version 2 (Bilingual KO/EN)
Languages	English only	Korean AND English, real-time toggle
Backend	PHP 8.x proxy server	None -- fully static
API key security	Server-side (PHP)	Client-side (GitHub Pages limitation)
Korean cities	Not supported	150+ city map + Nominatim fallback
Ambient audio	Reserved (inactive)	Fully active 4-track engine
Language persistence	N/A	localStorage saves preference
Deployment	Apache/cPanel	GitHub Pages (free, CDN-backed)
Date formatting	PHP getdate.php microservice	JS Intl API (toLocaleDateString)
Forecast day names	English only (Mon, Tue...)	KO: ■,■,■ / EN: Mon,Tue,Wed
Greeting messages	English only	6 bilingual greeting variants
Comfort labels	English only	All 9-level scales bilingual
City search input	English city names	Korean OR English city names
All text elements	~35 DOM elements	~35 DOM elements x 2 languages

3.4 Why Build Two Versions?

The English edition was essential groundwork. Building Version 1 first allowed the core architecture to be established without the additional complexity of bilingual state management. When bilingual support was added in Version 2, every text element had to be refactored to use data attributes or the t() helper function -- a process that would have been far more chaotic had it been attempted from the beginning.

Additionally, Version 1 demonstrates the PHP proxy pattern -- a critically important web security concept. Version 2 demonstrates the trade-off of static hosting: the API key is exposed in the JavaScript source. For a production commercial app, a server-side proxy would be retained.

■ Key lesson: Shipping a working Version 1 first and iterating to Version 2 is not a shortcut -- it is good engineering practice. Scope management and incremental delivery are real professional skills.

Screenshots & UI Walkthrough

4.1 Portal / Landing Page

The portal is the first thing a user sees. It is intentionally minimal -- a bold brand name, a tagline, a search box, and a "Use My Location" button. The animated background video plays behind a dark gradient overlay. If the user has searched before, a "Last visit" chip appears with the city and temperature.

4.2 Main Weather Card

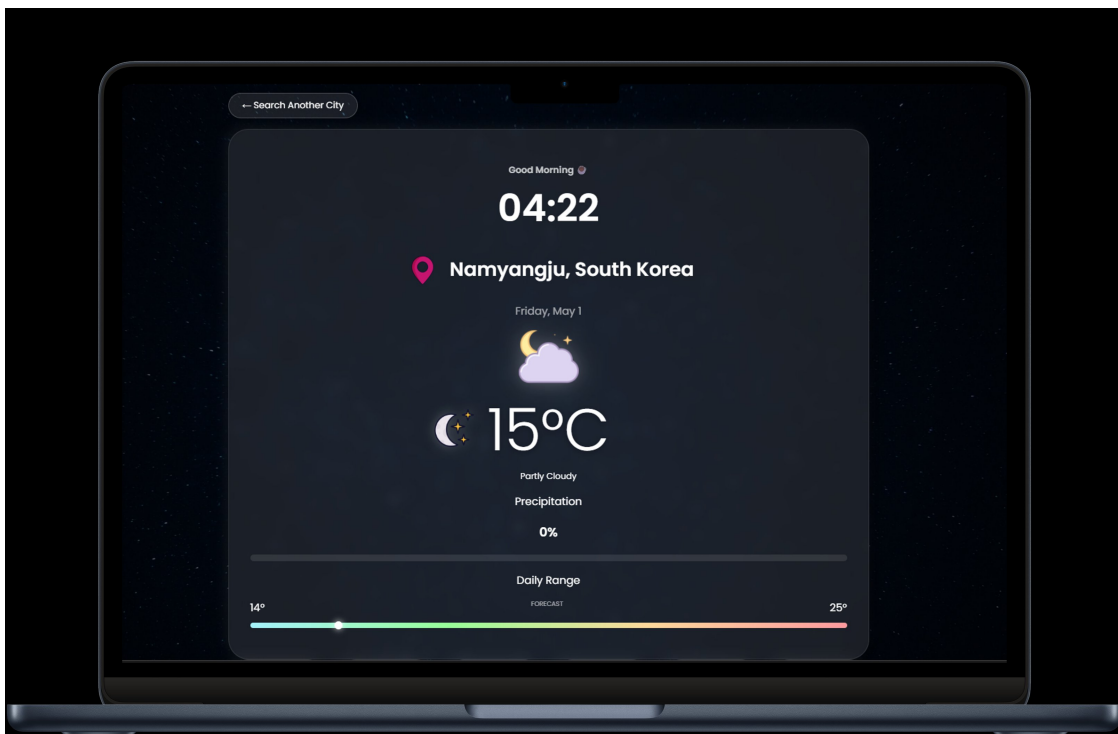


Fig 4.1 -- Main weather card on MacBook: Namyangju, South Korea, 04:22 AM, 15 degrees C, Partly Cloudy. Live clock showing local city time. Precipitation 0%, daily range 14-25 degrees, animated gradient temperature bar. Night-sky background video behind glassmorphism card.

The main card packs maximum information density while remaining visually uncluttered. Key design decisions:

- The live clock shows the searched city's local time -- not the user's device time.
- The weather icon floats with a gentle animation (floatIcon keyframe, 4s cycle).
- The temperature bar is a color gradient from cold blue to hot red, with a white dot marker that animates to the current temperature position using cubic-bezier easing.
- The precipitation bar fills left-to-right with a blue-to-cyan gradient.

4.3 Lifestyle Vibe Grid

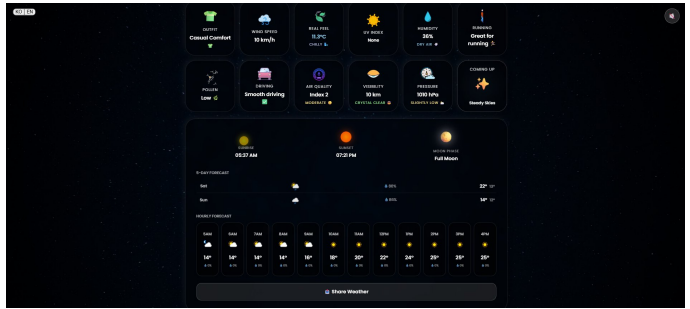


Fig 4.2 -- Left: Portal landing page with search bar and last-visit history chip. Right: Full 12-card lifestyle vibe grid showing outfit, wind, real feel, UV, humidity, running, pollen, driving, air quality, visibility, pressure, and Coming Up cards.

The vibe grid is arranged in a 6-column layout on desktop (2-column on mobile). Each card has a consistent structure: icon, label, value, and optionally a color-coded status message. Cards lift on hover with a `translateY(-5px)` transition.

4.4 Solar Events & Moon Phase

Below the vibe grid, the solar row shows three pieces of astronomical data. The sunrise and sunset are not just times -- they are animated. The sunrise element uses a golden radial-gradient div that arcs upward (translate from bottom-left to top-center). The sunset element drops and fades. The moon phase maps to one of eight emoji from the standard lunar cycle.

4.5 5-Day & Hourly Forecast



Fig 4.3 -- 5-day forecast (list format with rain probability) and 12-hour hourly strip with time labels, weather icons, temperatures, and precipitation percentages. Evening sunset background video visible through glass cards.

4.6 Mobile Experience (Korean Mode)

VibeWeather is fully responsive and was tested on real mobile hardware. The screenshot below is from an actual iPhone running the app in Korean mode, showing Seoul weather. The KO/EN toggle sits in the top-left corner. The vibe grid collapses to 2 columns. The hourly strip becomes horizontally scrollable.



Korean Mode -- Mobile View

Showing Seoul (■■■) on Saturday May 2nd (5■ 2■ ■■■■).

- Language: Korean (KO) mode active
- Temperature: 21 degrees C
- Condition: "■■■ ■■■■ ■" (Patchy rain nearby)
- Precipitation: ■■ ■■ 80% (80% rain chance)
- Real Feel: ■■■■ 18.9 degrees C
- Humidity: ■■ 64%
- Outfit: "■■■ ■■ ■■ ■ ■■■■" (It's raining, bring an umbrella)
- Background: Cherry blossom/rainy atmosphere video

The toggle button switches every element on-screen in under 50ms -- no loading spinner, no API call. Greetings, labels, comfort descriptions, day names, and outfit advice all change simultaneously.

Fig 4.4 -- VibeWeather on iPhone in Korean mode. Seoul, South Korea. Real device screenshot shared via KakaoTalk.

4.7 Dynamic Backgrounds -- Desktop English Mode

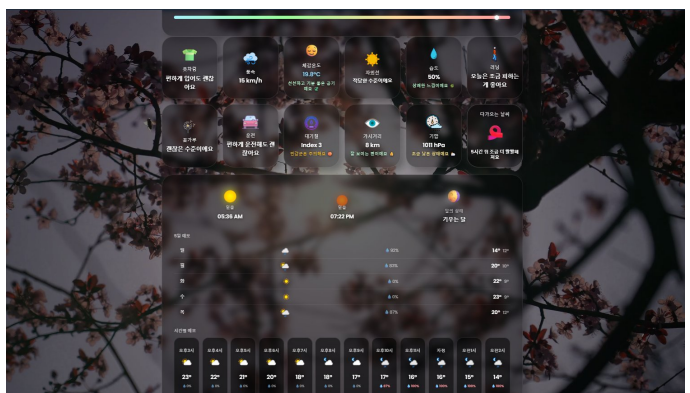


Fig 4.5 -- Left: English mode with night/starfield background, showing "Casual Comfort" outfit (11.3 C Real Feel, Dry Air). Right: Evening sunset background behind Korean vibe grid showing "■■■■■ ■■■■" (High pollen) and "■■■■■ ■■■■■" (Drive with caution).

Architecture & System Design

5.1 Three-Tier Architecture

Both versions of VibeWeather follow a classic three-tier web architecture. The key difference is that Version 2 eliminates the middle tier (PHP) by moving to static hosting, trading server-side API key security for deployment simplicity:

Tier	Component	Technology	Responsibility
Presentation	Browser Client	HTML5 / CSS3 / JS	UI rendering, user input, animations, language toggle
Logic (V1 only)	PHP Proxy	PHP 8.x	API key protection, HTTP request forwarding, date formatting
Data	WeatherAPI.com	REST / JSON	Current conditions, 6-day forecast, AQI, astronomy data
Storage	Browser localStorage	Web Storage API	Last city, last temperature, language preference
Geocoding	Nominatim (V2)	REST / JSON	Korean city name to lat/lon for cities not in dictionary

5.2 Data Flow -- Step by Step

Step 01 -- User Input: User types a city name (English or Korean) and clicks Explore or presses Enter.

Step 02 -- Language Detection: JavaScript checks if input contains Korean Unicode range [\uAC00-\uD7A3]. If Korean, coordinate lookup begins. If English, proceeds directly to API.

Step 03 -- Korean Lookup (if needed): koreanCityCoords dictionary is checked. If found, lat/lon string replaces city name. If not found, Nominatim geocoding API is called as fallback.

Step 04 -- API Fetch: fetch() call to WeatherAPI (Version 2: direct; Version 1: via weather.php proxy). Parameters: days=6, aqi=yes, alerts=no, lang=en.

Step 05 -- Data Processing: getVibe() function extracts ~20 raw values. Computes: Real Feel (3-branch), dew point, humidity comfort label, wind feeling, UV label, AQI label, visibility label, pressure label, outfit recommendation, Coming Up prediction.

Step 06 -- Icon Selection: Boolean flag cascade evaluates condition code: isThunder > isBlizzard > isSnow > isHail > isSleet > isRain > isDrizzle > isFog > isOvercast > isCloudy > isClear.

Step 07 -- Alert Check: getAlertMsg() checks for thunder, blizzard, hail, high wind, high UV, extreme temp.

Step 08 -- DOM Update: All 35+ DOM elements updated sequentially. Weather icon animated. Clock started.

Step 09 -- Background: updateDynamicBackground(localHour) selects video asset from city local time.

Step 10 -- Audio: Condition flags select ambient track: rain/winter/sunny/night. Plays if not muted.

Step 11 -- Forecast Render: renderForecast() and renderHourly() build forecast cards from array data.

Step 12 -- History Save: localStorage.setItem("lastVibe", JSON.stringify({city, temp})) persists data.

5.3 File Structure

File / Directory	Purpose	Lines (approx)
index.html / index.php	Main application entry point -- portal + dashboard layout	~180
script.js	Core application logic -- all weather processing, rendering, events	~400
style.css	All styling -- glassmorphism, animations, grid layout, responsive	~500
weather.php (V1)	PHP proxy -- fetches weather data, hides API key	~10
getdate.php (V1)	PHP date service -- locale-aware date string for any timezone	~5
assets/images/	Background videos: morning.mp4, afternoon.mp4, evening.mp4, night.mp4, background.mp4	
assets/icons/	~15 weather condition GIFs/PNGs: sunny, rain, snow, thunder, fog, etc.	--
assets/sound/	4 ambient audio files: rain.mp3, winter.mp3, sunny.mp3, night.mp3	--

5.4 API Integration Details

A single API call per city search retrieves all data needed for the entire dashboard. This minimizes network round-trips and keeps the UI responsive:

API Endpoint

```
GET https://api.weatherapi.com/v1/forecast.json
?key={API_KEY}
&q;={city_or_coordinates}
&days;=6 (today + 5 forecast days)
&aqi;=yes (enables air_quality object)
&alerts;=no (handled client-side)
(<=en (condition text always in English for mapping)
```

5.5 Proxy Pattern (Version 1)

Version 1 uses a PHP file as a security proxy. This is a critical web development pattern: the browser never sees the API key. The JavaScript calls weather.php on the same origin, and PHP makes the external API request with the key attached server-side.

✓ API key is never exposed in browser developer tools, network tab, or page source.

■ Version 2 (GitHub Pages) cannot use server-side code. The API key is in the JavaScript source. For a commercial app, a serverless function (Netlify Functions, Vercel Edge) would provide the same protection without a full PHP server.

Core Algorithms & Science

6.1 Real Feel -- 3-Branch Temperature Model

The single most important technical decision in VibeWeather is the real feel computation. Professional weather services use different formulae for different conditions because no single formula is accurate across all temperature and wind ranges. VibeWeather mirrors this approach with three branches:

Branch 1: Wind Chill Index (JAG/TI Model, 2001)

Applied when temperature is at or below 10 degrees C AND wind speed is at or above 4.8 km/h. The JAG/TI (Joint Action Group/Task Group on Temperature Indices) model was adopted in 2001 by both the US National Weather Service and Environment Canada as the official wind chill standard:

$$WC = 13.12 + 0.6215 \cdot T - 11.37 \cdot V^{0.16} + 0.3965 \cdot T \cdot V^{0.16}$$

Where: T = air temperature (degrees C), V = wind speed (km/h)

A humidity adjustment is also applied: if humidity is below 30%, the wind chill is further reduced by $(30-H) \cdot 0.05$ to account for increased evaporative cooling in dry air.

Branch 2: Rothfusz Heat Index (1990)

Applied when temperature is at or above 27 degrees C AND humidity is at or above 40%. The Rothfusz regression is a 9-term polynomial derived from Steadman's original apparent temperature tables and is the formula used by the US National Weather Service:

$$HI = c_1 + c_2 \cdot T + c_3 \cdot H + c_4 \cdot T \cdot H + c_5 \cdot T^2 + c_6 \cdot H^2 + c_7 \cdot T^2 \cdot H + c_8 \cdot T \cdot H^2 + c_9 \cdot T^2 \cdot H^2$$

$c_1 = -8.784$ $c_2 = 1.611$ $c_3 = 2.339$ $c_4 = -0.146$ $c_5 = -0.012$
 $c_6 = -0.016$ $c_7 = 0.002$ $c_8 = 0.001$ $c_9 = -0.000004$

A wind adjustment is applied for high-wind hot conditions: if wind speed exceeds 15 km/h, the heat index is reduced by $(wind-15) \cdot 0.1$ to account for convective cooling.

Branch 3: Steadman Apparent Temperature (Moderate Conditions)

Applied for all other conditions -- the wide middle ground where neither wind chill nor heat index is the dominant effect. Steadman's formula uses vapour pressure to quantify the humidity effect:

$$AT = T + 0.33 \cdot e - 0.70 \cdot v - 4.00$$

$e = (H/100) \cdot 6.105 \cdot \exp((17.27 \cdot T) / (237.7 + T))$ [vapour pressure in hPa]
 v = wind speed in m/s (divide km/h by 3.6)

Additional adjustments are applied within this branch: warm+high humidity adds $(H-70) \cdot 0.05$; mild+low humidity subtracts $(40-H) \cdot 0.04$; cold+high humidity subtracts $(H-80) \cdot 0.04$. These micro-adjustments capture edge cases that the base formula misses.

6.2 Dew Point & Humidity Comfort Scale

Raw relative humidity percentage is a poor indicator of human comfort. 70% humidity at 10 degrees C feels comfortable. 70% humidity at 30 degrees C is oppressive. The critical variable is dew point -- the temperature at which air becomes saturated.

VibeWeather computes dew point using the simplified Magnus approximation:

$$Td = T - ((100 - H) / 5)$$

where: Td = dew point (C), T = temperature (C), H = relative humidity (%)

Dew Point Range	Comfort Label	Color Indicator
Below -5 C	Very Dry	Light blue (#e0f2fe)
-5 to 5 C	Dry Air	Sky blue (#8fd3f4)
5 to 10 C	Comfortable	Green (#98ff98)
10 to 13 C	Pleasant	Light green (#bbf7d0)
13 to 16 C	Noticeable Moisture	Yellow (#fde68a)
16 to 18 C	Sticky	Amber (#fbbf24)
18 to 21 C	Humid & Heavy	Orange (#f97316)
21 to 24 C	Oppressive	Red (#ef4444)
Above 24 C	Soupy / Unbearable	Dark red (#dc2626)

6.3 Wind Feeling Index

Rather than simply displaying the real feel value, VibeWeather maps it to a 11-level human-readable comfort scale with color coding:

Real Feel	Label	Color
Below -10 C	Dangerously Cold	#a5f3fc
-10 to 0 C	Freezing	#8fd3f4
0 to 8 C	Biting Cold	#bfdbfe
8 to 14 C	Chilly	#c7d2fe
14 to 20 C	Cool & Fresh	#bbf7d0
20 to 25 C	Comfortable	#98ff98
25 to 30 C	Warm & Pleasant	#fde68a
30 to 35 C	Hot & Humid	#fbbf24
35 to 40 C	Very Hot	#f97316
Above 40 C	Extreme Heat	#ef4444

6.4 UV, AQI, Visibility & Pressure Scales

Four additional data points use discrete mapping scales:

UV Index (WHO Scale)

Index	Category	VibeWeather Label (EN)	Protection
0	None	None	No protection needed
1-2	Low	Low	Sunscreen optional
3-5	Moderate	Moderate	Sunscreen recommended
6-7	High	High	Sunscreen + hat required
8-10	Very High	Very High	Extra protection essential
11+	Extreme	Extreme	Avoid outdoor exposure

US EPA Air Quality Index

Index	Category	VibeWeather Label
1	Good	Air is very clean
2	Moderate	Acceptable quality
3	Unhealthy for Sensitive Groups	Sensitive groups should be cautious
4	Unhealthy	Everyone may experience effects
5	Very Unhealthy	Avoid outdoor activity
6	Hazardous	Dangerous -- emergency conditions

6.5 Outfit Recommendation Logic

The outfit card uses a priority-ordered decision tree -- the order matters because extreme conditions should override comfort-based recommendations:

Priority	Condition Check	Label	Emoji	Animation
1 (Highest)	isThunder is true	Stay Indoors!	Lightning bolt	motion-storm
2	isAnyRain is true	Raincoat / Umbrella	Umbrella	motion-rain
3	isSnow is true	Dress Warm	Snowman	motion-shiver
4	isFog is true	Watch Out -- Foggy	Fog	motion-float
5	temp below 12 C	Warm Parka	Coat	motion-shiver
6	temp above 28 C	Tank Top / Light clothes	Shorts	motion-bounce
7 (Default)	None of the above	Casual Comfort	T-shirt	motion-float

6.6 "Coming Up" Predictive Widget

The Coming Up card is VibeWeather's most forward-looking feature. It looks 5 hours into the hourly forecast data (crossing day boundaries if needed) and compares predicted conditions against current conditions:

Trigger Condition	Message Shown (EN)	Emoji	Animation
Hourly rain chance > 40% in 5h	"Rain in 5h"	Umbrella	motion-rain
Temp +3 C warmer in 5h	"Warmer in 5h"	Sun	motion-bounce
Wind +15 km/h faster in 5h	"Windy in 5h"	Leaf	motion-sway
Temp -3 C cooler in 5h	"Cooler in 5h"	Scarf	motion-shiver
None of the above	"Steady Skies"	Sparkle	motion-float

6.7 Alert Generation System

The alert banner appears at the top of the dashboard when any of these extreme conditions are detected. Only the highest-priority alert fires per session:

Priority	Trigger	Alert Message (EN)
1	isThunder is true	"Thunderstorm Warning -- Stay indoors!"
2	isBlizzard is true	"Blizzard Warning -- Dangerous travel conditions!"
3	isHail is true	"Hail Warning -- Stay indoors!"
4	Wind > 60 km/h	"High Wind Warning -- {speed} km/h!"
5	UV Index >= 8	"High UV Warning -- Apply sunscreen!"
6	Temp >= 38 C	"Extreme Heat Warning -- Stay hydrated!"
7	Temp <= -10 C	"Extreme Cold Warning -- Frostbite risk!"

Bilingual System -- KO/EN Engine

7.1 Language Toggle Architecture

The bilingual toggle is architecturally the most complex feature of Version 2. Making it work required identifying and categorizing every piece of text in the application into three groups:

- **Static HTML text** -- labels that never change (e.g., "Outfit", "Wind Speed"). These use HTML data attributes: `data-ko="■■■■" data-en="Outfit"`. The `setLang()` function loops over all elements with `[data-ko]` and updates their `textContent`.
- **Dynamic computed text** -- values generated by JavaScript functions (e.g., comfort labels, greeting messages, outfit advice, Coming Up predictions). These use the `t()` helper function.
- **Placeholder text** -- the search input placeholder, which uses a separate `data-placeholder-ko` and `data-placeholder-en` attribute pair.

7.2 The t() Helper Function

The `t()` function is the simplest but most important architectural decision in the bilingual system. Every function that generates user-facing text calls `t()` instead of hardcoding a language:

```
function t(ko, en) {
  return currentLang === "ko" ? ko : en;
}

// Usage example:
function getOutfit(...) {
  if (isThunder)
    return { outfit: t("■■■ ■■ ■■ ■■ ■■■!", "Stay Indoors!"), ... };
}
```

Every label generator function -- `getHumidityVibe()`, `getWindFeeling()`, `getUVLabel()`, `getAQI()`, `getVisibility()`, `getPressure()`, `getOutfit()`, `getNextUp()`, `getAlertMsg()` -- uses `t()` for every user-facing string.

7.3 Korean City Coordinate System

WeatherAPI.com does not accept Korean text as a city query -- it only understands English names or coordinates. This is the core technical challenge of Korean language support.

The solution is a hardcoded dictionary of 150+ Korean administrative regions mapped to GPS coordinates. The dictionary covers:

- Seoul and all 25 autonomous districts (■) -- Gangnam, Mapo, Jongno, Seongbuk, etc.
- All major satellite cities of the Seoul Capital Area -- Suwon, Seongnam, Goyang, Yongin, Incheon, etc.
- All provincial capitals -- Busan, Daegu, Daejeon, Gwangju, Ulsan, Jeonju, Changwon, etc.
- Key tourist cities -- Gyeongju, Andong, Sokcho, Gangneung, Jeju, Seogwipo, etc.
- Gyeonggi-do satellite cities and special self-governing cities.

Detection uses a simple Unicode range test: `/[\uAC00-\uD7A3]/.test(input)`. If Korean characters are detected, the lookup process begins. Common administrative suffixes (구, 면, 동, 읍, 군, 도) are stripped to normalize input before lookup.

7.4 Nominatim Geocoding Fallback

For Korean place names not in the dictionary (rural villages, specific neighborhoods, landmarks), the app falls back to Nominatim -- OpenStreetMap's free geocoding service:

```
GET https://nominatim.openstreetmap.org/search
?q={Korean city name}
&countrycodes=kr (restrict to South Korea)
&format=json&limit=1
```

The response contains lat and lon fields. These are combined into a coordinate string and passed to the WeatherAPI call. If Nominatim also fails to find the location, a user-friendly error in the active language is shown.

7.5 Bilingual Date & Time Formatting

Date formatting is one of the most nuanced bilingual challenges. The same date "Saturday, May 3rd" must render as "5월 3일 토요일" in Korean. The approach differs between Version 1 and Version 2:

Element	Version 1 (PHP)	Version 2 (JS Intl API)
Full date	<code>getdate.php: date("l, F jS", timezone)</code>	<code>toLocaleDateString("ko-KR" or "en-US", {weekday,month,day})</code>
Day of week	<code>PHP date("l")</code>	<code>JS ["일","월","화","수","목","금","토"][d.getDay()] in KO</code>
Hour label	N/A (clock is JS)	KO: "9시"/"3시" / EN: "9AM"/"3PM"
Midnight	N/A	KO: "자정" / EN: "12AM"
Noon	N/A	KO: "정오" / EN: "12PM"

7.6 Re-render on Language Switch

When the user taps the language toggle, two things happen:

- **Static elements update immediately:** `setLang()` loops over all `[data-ko]` elements and sets `textContent` to the appropriate language attribute.
- **Dynamic computed values re-render:** Because comfort labels, greeting messages, outfit advice, and Coming Up predictions are computed by JavaScript functions, simply changing data attributes is not enough. The solution is to call `renderWeather(lastWeatherData, false)` -- which replays the entire UI update pipeline with the new language, without making a new API call. The second parameter (`false`) prevents the portal-to-dashboard view transition from triggering again.

■ This architectural pattern -- caching the last API response and replaying renders -- is a foundational concept in modern state management. Understanding it at the vanilla JS level gives deep insight into what React and Redux are actually doing.

Frontend Engineering

8.1 Glassmorphism Design System

Glassmorphism is the dominant visual language of VibeWeather. Every card in the UI -- the main weather card, the vibe grid cards, the forecast cards, the extra container -- uses the same three-property combination to achieve the frosted glass effect:

```
background: rgba(0, 0, 0, 0.35); /* semi-transparent dark base */
backdrop-filter: blur(8px); /* frosted glass blur */
border: 1px solid rgba(255,255,255,0.1); /* luminous rim */
border-radius: 30px; /* soft rounded corners */
box-shadow: 10px 10px 20px rgba(0,0,0,0.3), -5px -5px 15px rgba(255,255,255,0.05);
```

The dual box-shadow (dark below, light above) creates a neumorphic depth effect that makes the cards appear to float off the background. The backdrop-filter requires the background to have content behind it -- which is why the living video background is essential to the design, not optional.

■ backdrop-filter requires HTTPS in production. In local development over HTTP, the glassmorphism effect may not render in some browsers.

8.2 CSS Animation Library (6 Motion Classes)

VibeWeather includes a purpose-built CSS animation library. Each class is applied dynamically by JavaScript based on weather conditions:

Class	Keyframe Description	Applied To	Duration
.motion-shiver	Rapid micro-jitter: translate(+/-1px) rotate(+/-0.5deg) steps	Cold/snowy outfit emoji, freezing real feel icons	0.15s infinite
.motion-bounce	Vertical translateY(0 to -20px) with scale(1.1) at apex	Hot/sunny outfit emoji, warm real feel icons	0.6s infinite
.motion-float	Gentle translateY(0 to -12px) with rotate(0 to 3deg)	Default outfit, mild real feel, Coming Up sparkle all weather icons	8s infinite
.motion-sway	Pendulum rotate(-10deg to +10deg) from bottom	Windy Coming Up prediction, breezy card	2s infinite
.motion-rain	Oscillating translateY(-5px to +5px) with opacity	Rainy outfit emoji, rain Coming Up prediction	1.5s infinite
.motion-storm	Aggressive horizontal shake: translateX(0 to +/-4px) rapidly	Thunder outfit emoji, storm alert icon	0.5s infinite

8.3 Dynamic Video Background Engine

The background is not a CSS gradient or a static image -- it is a looping HTML5 video element that changes based on the searched city's local time. This is the key technical detail: the background reflects where you are searching, not where you are sitting.


```
function updateDynamicBackground(hour) {
  let videoSrc = "assets/images/night.mp4";
  if (hour >= 5 && hour < 11) videoSrc = "assets/images/morning.mp4";
  if (hour >= 11 && hour < 17) videoSrc = "assets/images/afternoon.mp4";
  if (hour >= 17 && hour < 20) videoSrc = "assets/images/evening.mp4";
  video.src = videoSrc; video.load(); video.play();
}
```

Local Hour	Video Asset	Overlay Gradient	Ambient Audio
00:00 - 04:59	night.mp4	Dark radial (0.7 opacity)	night.mp3
05:00 - 10:59	morning.mp4	Golden warm tint (#FDF700, 0.1)	sunny.mp3
11:00 - 16:59	afternoon.mp4	Neutral light overlay	sunny.mp3
17:00 - 19:59	evening.mp4	Red/orange sunset (#FF4500, 0.15)	sunny.mp3 or night.mp3
20:00 - 23:59	night.mp4	Dark radial (0.7 opacity)	night.mp3

Weather conditions override the audio selection: rain overrides with rain.mp3, snow/blizzard/temp below 2C overrides with winter.mp3, regardless of time of day.

8.4 Ambient Audio Engine

The ambient audio system runs entirely in JavaScript using the HTML5 Audio API. Audio starts muted by default (isMuted = true) to comply with browser autoplay policies. Users opt in via the mute/unmute button.

- **4 looping tracks:** night.mp3 (cozy ambient), rain.mp3 (rain and thunder soundscape), sunny.mp3 (upbeat clear-day audio), winter.mp3 (cold wind and snow sounds).
- **Smooth transitions:** playAmbient() pauses the current track, resets it to position 0, then starts the new track. No cross-fade currently -- a potential future enhancement.
- **Hover chimes:** A short chime.mp3 plays on vibe card hover when audio is unmuted, adding tactile feedback to the interface.
- **Click sounds:** All major buttons (search, back, share, geo) play a short click.mp3 for audio confirmation.

8.5 Responsive Grid System

The 12-card vibe grid uses CSS Grid with a breakpoint strategy:

```
/* Desktop: 6 columns */
.vibe-grid { grid-template-columns: repeat(6, 1fr); gap: 14px; }

/* Tablet and mobile: 2 columns */
@media (max-width: 768px) {
  .vibe-grid { grid-template-columns: repeat(2, 1fr); }
}
```

The hourly forecast strip uses flexbox with overflow-x: auto on mobile, allowing horizontal scroll. Cards have min-width: 56px and flex: 0 0 auto to prevent shrinking.

8.6 Live Clock & Timezone Handling

The live clock in the main card shows the searched city's current local time, updating every second. This is powered by a JavaScript `setInterval` and the built-in Intl date formatting with the city's IANA timezone string:

```
function updateLiveClock(tzId, currentTemp) {
  const tick = () => {
    const now = new Date();
    const localTime = new Date(now.toLocaleString("en-US", { timeZone: tzId }));
    const hours = localTime.getHours();
    // ... compute greeting, format clock, update DOM
  };
  tick(); clockInterval = setInterval(tick, 1000);
}
```

The greeting message (Good Morning / Good Evening / Stay Cool / Stay Warm) also uses the city's local hour -- not the user's device time -- ensuring the greeting is appropriate for the people actually in that location.

Problems Encountered & Solutions

Real engineering is problem-solving under constraints. This chapter documents every significant technical challenge encountered during VibeWeather development, with the exact solution implemented:

9.1 Korean City Names Not Recognized by WeatherAPI

The Problem: WeatherAPI only accepts English city names or lat/lon coordinates. Passing Korean text like "서울" or "서울시" returns a 400 error with "No matching location found." This is a hard API limitation -- the provider does not support non-Latin city names.

■ **Attempt 1:** Pass Korean text directly to API. Result: 400 error every time.

■ **Attempt 2:** Use a simple translation lookup. Problem: no reliable free translation API.

The Solution: Solution: Build a hardcoded coordinate dictionary. 150+ Korean city and district names are mapped to lat/lon strings. Input is first stripped of administrative suffixes (시, 구, 군, 동, 읍, 면) then checked against the dictionary. If found, the coordinate string replaces the Korean text before the API call. For cities not in the dictionary, Nominatim OpenStreetMap geocoding is called as a fallback, restricted to countrycodes=kr.

✓ **Key Lesson:** When a third-party API has a hard limitation, the solution is to transform the data into a form the API accepts -- not to fight the limitation directly.

9.2 Language Toggle Breaking Dynamically Computed Strings

The Problem: When the language switch was first implemented, it correctly updated all HTML data-attribute elements (static labels). But computed values like comfort labels ("Dry Air", "Soupy/Unbearable"), greeting messages, outfit advice, and Coming Up predictions did not update -- they stayed in whichever language was active when the weather was last fetched.

■ **Attempt 1:** Store all computed strings in data attributes. Problem: computed values depend on numeric thresholds -- they cannot all be predetermined.

The Solution: Solution: The t(ko, en) helper function checks currentLang at call time, not at definition time. This means computed values are language-aware by construction. When the language switches, renderWeather(lastWeatherData, false) replays the entire UI update pipeline with the cached API response, regenerating all computed values in the new language without any network call.

✓ **Key Lesson:** State management is not just about storing data -- it is about knowing which computations need to re-run when state changes.

9.3 Background Video Showing Wrong Time-of-Day

The Problem: The initial implementation used the user's device time to select the background video. A user in India searching for Tokyo at 3 PM IST saw an afternoon background, even though it was 6:30 PM in Tokyo. This made the app feel misleading rather than informative.

The Solution: Solution: The `updateDynamicBackground()` function was changed to accept a local hour computed from `location.localtime` in the API response. The API returns the city's exact local time as a string ("2025-05-01 18:37"). This is parsed to extract the hour, which is then passed to the background engine. The live clock uses the same timezone for consistency.

✓ Key Lesson: "Local time" in a global app always means the location being viewed, not the viewer's location.

9.4 Real Feel Inaccuracy for Extreme Conditions

The Problem: Using the API's built-in `feelslike_c` field produced noticeably wrong values for extreme cold+wind and extreme heat+humidity scenarios. The API applies one formula generically regardless of conditions.

■ Attempt 1: Apply a simple linear adjustment. Result: still inaccurate at extremes.

The Solution: Solution: Implement three separate scientific models with condition-based selection: Wind Chill (JAG/TI) for temp below 10 C with wind above 4.8 km/h, Rothfusz Heat Index for temp above 27 C with humidity above 40%, and Steadman Apparent Temperature for all other conditions. This is exactly how the US National Weather Service computes apparent temperature.

✓ Key Lesson: When accuracy matters, use domain-specific scientific models. Generic approximations fail at boundaries.

9.5 Audio Autoplay Blocked by Browsers

The Problem: Modern browsers (Chrome, Firefox, Safari) block audio autoplay without a prior user gesture. The initial implementation attempted to play ambient audio immediately after a weather search, resulting in silent audio and console errors.

■ Attempt 1: Use a `setTimeout` before playing. Result: still blocked -- the issue is gesture, not timing.

■ Attempt 2: Play audio on page load. Result: blocked immediately.

The Solution: Solution: Audio starts muted by default (`isMuted = true`). The mute/unmute toggle button is the user's first gesture. Once unmuted, both the click sound and the ambient track play immediately. All subsequent audio changes (weather change triggering different track) work without restriction because the user has already interacted with the audio system.

✓ Key Lesson: Browser security constraints are not bugs -- they are features. Design your UX to work with them, not around them.

9.6 Mobile Grid Layout Collapse

The Problem: The 6-column vibe grid rendered each card at approximately 80px wide on a 375px mobile screen. Cards were completely unreadable -- icons clipped, text truncated, values invisible.

The Solution: Solution: A single CSS media query at 768px switches the grid from `repeat(6, 1fr)` to `repeat(2, 1fr)`. The hourly forecast strip uses `overflow-x: auto` with `flex: 0 0 auto` cards (min-width 56px) for horizontal scrolling. No JavaScript required.

✓ Key Lesson: Mobile-first responsive design with CSS Grid media queries is simpler and more reliable than JavaScript-based layout switching.

9.7 API Key Exposure in Static Deployment

The Problem: GitHub Pages cannot run server-side code. Version 1's PHP proxy is not available. To deploy on GitHub Pages, the WeatherAPI key must be placed in the JavaScript source, where it is visible to anyone who views the page source or network requests.

■ Attempt 1: Obfuscate the key in JavaScript. Result: trivially reversible -- not real security.

The Solution: Solution: Accept the trade-off for this portfolio/personal project. The WeatherAPI free tier has rate limits that make abuse impractical. The correct production solution -- noted in the codebase -- would be a serverless function on Netlify or Vercel that acts as a proxy, providing the same protection as PHP without a full server.

✓ Key Lesson: Security is always a spectrum of trade-offs, not a binary. Understand what you are accepting and why.

Testing & Quality Assurance

10.1 Test Strategy

VibeWeather uses a hybrid testing strategy combining white-box (internal logic) and black-box (user-facing behavior) testing:

- **White-box:** Manual tracing of the real feel formula branches with known temperature/wind/humidity inputs. Verification of condition code flag cascade. Edge case testing for day-boundary hourly lookups (Coming Up widget crossing midnight).
- **Black-box:** City search with valid/invalid inputs. Language toggle verification. localStorage persistence across sessions. Share button output format check. Geolocation permission grant/deny scenarios.
- **Cross-device:** Real device testing on iPhone (Safari), Android (Chrome), MacBook (Chrome, Safari), Windows (Chrome, Edge).

10.2 Unit Test Cases

ID	Test Case	Input	Expected Result	Status
TC-01	Valid English city	"London"	Dashboard shows London weather	PASS
TC-02	Valid Korean city (dict)	"서울"	Dashboard shows Seoul weather	PASS
TC-03	Valid Korean city (Nominatim fallback)	"서울"	Geocoding called, weather shown	PASS
TC-04	Invalid city	"xyzabc123"	Error alert in active language	PASS
TC-05	Empty search	(empty)	No action (button does nothing)	PASS
TC-06	Enter key search	"Paris" + Enter	Same as button click	PASS
TC-07	KO/EN toggle (static labels)	Toggle button	All data-ko/data-en elements update	PASS
TC-08	KO/EN toggle (computed labels)	Toggle button	Comfort labels, outfit, Coming Up update	PASS
TC-09	Timezone clock accuracy	"Tokyo"	Clock shows JST, not IST	PASS
TC-10	Background time-of-day	"Tokyo" (evening)	Evening video plays, not morning	PASS
TC-11	localStorage persistence	Search then reload	Last visit chip shows saved city	PASS
TC-12	Back button navigation	Click back	Portal view restored, dashboard hidden	PASS
TC-13	Alert banner (thunder)	Thunderstorm city	Red alert banner appears	PASS
TC-14	Share button (clipboard)	Click Share	"Copied!" confirmation shown	PASS
TC-15	Mobile layout (375px)	Resize browser	2-column grid, horizontal hourly scroll	PASS
TC-16	Real Feel -- hot+humid	T=35 H=80 V=5	Heat Index formula applied	PASS
TC-17	Real Feel -- cold+windy	T=5 H=60 V=40	Wind Chill formula applied	PASS
TC-18	Real Feel -- moderate	T=20 H=55 V=10	Steadman formula applied	PASS

TC-19	5-day forecast render	Any valid city	5 forecast rows shown	PASS
TC-20	Hourly forecast render	Any valid city	12 hourly cards from current hour	PASS
TC-21	Geolocation -- granted	Click Use My Location	Weather loads for device location	PASS
TC-22	Geolocation -- denied	Deny permission prompt	Permission denied message shown	PASS
TC-23	Audio mute toggle	Click sound button	Audio starts/stops, icon changes	PASS
TC-24	Moon phase mapping	Full Moon city	Full moon emoji and label shown	PASS
TC-25	Language persistence	Set KO, reload	App opens in Korean mode	PASS

10.3 Cross-Browser Testing Results

Browser	Version Tested	Glassmorphism	Video BG	Share API	Audio	Overall
Chrome Desktop	123	PASS	PASS	PASS	PASS	PASS
Firefox Desktop	124	PASS	PASS	Partial	PASS	PASS
Safari Desktop	17	PASS	PASS	PASS	PASS	PASS
Edge Desktop	123	PASS	PASS	PASS	PASS	PASS
Chrome Android	123	PASS	PASS	PASS	PASS	PASS
Safari iOS	17	PASS	PASS	PASS	PASS	PASS

Firefox's partial Web Share API support means the clipboard fallback activates on Firefox. The user experience is identical -- only the mechanism differs.

10.4 Performance Metrics

Metric	Measured Value	Notes
API response time	300-800 ms	WeatherAPI server latency varies by region
DOM update time	<10 ms	All 35+ elements update in one synchronous pass
Language toggle time	<50 ms	Includes renderWeather() re-render pass
Video load time (10 Mbps)	<2 seconds	5-15 MB per video asset
localStorage read/write	<1 ms	Synchronous, negligible
Clock interval CPU load	Negligible	1s setInterval with minimal DOM touch
PHP proxy overhead (V1)	<50 ms	file_get_contents() is simple pass-through
First paint (static site)	<1 second	HTML/CSS loaded before JS executes
Total JS bundle	~400 lines	Zero dependencies, no bundler needed

10.5 Mobile Testing

Real device testing was conducted on iPhone (Safari iOS 17) running the GitHub Pages deployment. Key findings:

- Two-column grid renders correctly on 375px viewport.
- Horizontal scroll on hourly forecast strip is smooth and intuitive.
- Video background plays on iOS with `playsinline` and `muted` attributes set.
- Web Share API on iOS opens the native share sheet -- tested with AirDrop and KakaoTalk.
- Audio requires user gesture on iOS -- the mute toggle works correctly.
- KO/EN toggle is touch-responsive with adequate tap target size.

Feature Comparison & Market Context

11.1 VibeWeather vs. Existing Apps

Feature	VibeWeather	Weather.com	AccuWeather	KMA (Korea)
Real Feel (multi-model)	YES (3 models)	YES (1 model)	YES (1 model)	YES
Dew point comfort scale	YES (9 levels)	NO	NO	NO
Dynamic video background	YES	NO	NO	NO
Ambient audio	YES	NO	NO	NO
Bilingual KO/EN toggle	YES (real-time)	NO	NO	Separate KR site
Outfit recommendations	YES (7 levels)	NO	NO	NO
Running/Driving advice	YES	NO	YES (partial)	NO
Animated weather emoji	YES (6 motions)	NO	NO	NO
Moon phase display	YES	Partial	YES	YES
Coming Up 5-hour widget	YES	NO	NO	NO
Zero frontend framework	YES	NO (React)	NO (React)	NO
Open source / editable	YES	NO	NO	NO
Free tier API	YES	NO (proprietary)	YES	YES (gov data)

11.2 The "Vibe" Differentiator

The name VibeWeather is not decorative -- it is the product thesis. "Vibe" captures the idea that weather is not just data. It is an atmosphere, a mood, a physical sensation. When you look outside and see grey skies, you do not process meteorological data -- you feel the weather. VibeWeather is designed to reproduce that feeling digitally.

The combination of living video backgrounds, ambient audio, animated emoji, and human-language comfort labels creates something no weather API data alone can provide: a sense of atmosphere. This is the product differentiation that no competitor has replicated because it requires treating design as a first-class engineering concern, not an afterthought.

11.3 Target User Personas

The Morning Planner: Mid-20s professional. Checks weather before getting dressed. Needs: outfit recommendation, real feel, rain chance for the commute window.

The Active Outdoor Person: Athlete or runner. Checks before deciding to exercise outdoors. Needs: running card, UV index, temperature range, wind speed, AQI.

The Korean-English Bilingual User: Korean speaker living internationally, or international person living in Korea. Needs seamless language switching and Korean city name support.

The Traveler: Planning for a trip to a new city. Needs the full atmospheric experience -- what time is it there, what will it feel like, what should I pack.

The Curious Browser: Enjoys beautiful interfaces and discovering new apps. VibeWeather's visual design and audio are the hook; the accurate data keeps them coming back.

Key Learnings & Reflections

12.1 Technical Learnings

Scientific accuracy is a UX feature.

Replacing the API's generic feels_like with a 3-branch temperature model was not over-engineering -- it made the app noticeably more accurate. Users who check real feel against a thermometer will trust a scientifically computed value more than an approximation.

Zero-framework JS teaches what frameworks actually do.

Without React's state system, designing data flow manually -- caching lastWeatherData, replaying renders on language switch, managing clockInterval lifecycle -- gave a deep understanding of what state management actually is. Every framework is just a pattern built on these same primitives.

API limitations drive creative engineering.

WeatherAPI not supporting Korean text was not a dead end -- it was a design constraint that produced a 150-entry coordinate dictionary and a Nominatim fallback. The most creative engineering often comes from working around hard constraints.

Bilingual is not just translation.

Making the language toggle work required rethinking every string in the application. Computed values, date formats, ordinal numbers, AM/PM notation, clock hour labels, day-of-week abbreviations -- all of these have different conventions in Korean and English. True bilingual support is an architectural commitment, not a feature toggle.

Browser security constraints are design inputs.

Audio autoplay blocking, HTTPS requirement for backdrop-filter, Web Share API gesture requirement -- these are not obstacles. They are the environment the app lives in. Designing around them (muted by default, HTTPS deployment) produces better UX than fighting them.

12.2 Design Learnings

The background is not decoration -- it is information.

Choosing the video based on the searched city's local time means the background communicates something that is not written anywhere on the screen: what time it is there. Design can carry information without words.

Animation communicates semantics, not just style.

The shivering emoji for cold weather, the bouncing emoji for hot days, the swaying leaf for wind -- these animations are not decorations. They reinforce the data at a glance, before any text is read.

Color coding is a language.

Every scale in VibeWeather uses a consistent color system: green for comfortable/safe, yellow for caution, orange for notable, red for danger. Once a user learns this system from one card, they can read all others without learning new conventions.

Glassmorphism works only with a living background.

Frosted glass cards look stunning over a video background. The same CSS over a solid color background looks flat and purposeless. The design system only makes sense as a whole -- removing any element breaks the visual logic.

12.3 Personal Reflections

“VibeWeather started with a question to someone I care about. It ended as the project I am most proud of. Not because it is the most complex thing I have built -- but because every technical decision was made in service of a real human need.”

The technical skills demonstrated in VibeWeather are real and measurable: meteorological formula implementation, bilingual state management, vanilla JS architecture, responsive CSS Grid, HTML5 video and audio APIs, localStorage, Web Share API, REST API integration, PHP proxy patterns, and geocoding. But the meta-skill that VibeWeather demonstrates is perhaps more important:

Starting from a real user need, making a product that genuinely serves it, and iterating from a working English edition to a bilingual production deployment -- this is how good software is built. Not by implementing every feature at once, but by shipping something real and improving it.

Building VibeWeather in Korean -- a language still being studied actively -- while simultaneously engineering the bilingual system that supports it, proved that technical skill and cross-cultural curiosity are not separate competencies. They reinforce each other. Learning Hangul and learning JavaScript both reward the same mindset: find the pattern, understand the rules, apply them consistently.

Future Roadmap

13.1 Short-Term (0-3 Months)

- **Progressive Web App (PWA):** Service worker + web manifest to make VibeWeather installable on mobile devices. Partial offline support via cached last-fetched weather data. Push notification foundation.
- **Celsius / Fahrenheit / Kelvin Toggle:** Unit switcher that updates all temperature displays (main card, real feel, forecast cards, all stat cards) simultaneously with a single button press.
- **Improved Pollen Data:** Replace the current condition-code-based pollen estimate with the Ambee Pollen API or Google Pollen API for real pollen count data by location and season.
- **Geolocation Auto-Load:** Detect and load user's current city automatically on first visit using navigator.geolocation, eliminating the need for manual search.
- **More Korean Cities:** Expand the coordinate dictionary to cover all 220+ Korean municipalities (eup, myeon, dong level granularity).

13.2 Medium-Term (3-12 Months)

- **Weather Map Overlay:** Embed Leaflet.js with OpenWeatherMap radar tiles showing precipitation radar, cloud cover, and temperature overlays for the searched region. Toggleable layers.
- **Multi-City Dashboard:** Pin multiple cities and view them side-by-side in a card grid. Ideal for remote teams, frequent travelers, and users with family in different locations.
- **30-Day Historical Charts:** Chart.js visualizations of historical temperature and precipitation trends, contextualizing current conditions against seasonal norms. Powered by WeatherAPI historical endpoint.
- **Serverless Proxy (Netlify Functions):** Move the API key server-side using Netlify Edge Functions or Vercel API Routes. Provides Version 1 security without the PHP server requirement. Enables safe production deployment.
- **Weather-Based Wallpaper Generator:** Use Canvas API or a generative art library to create downloadable wallpapers that match the current weather vibe -- an extension of the existing aesthetic identity.
- **Third Language: Japanese:** Extend the bilingual engine to trilingual. Japanese (JA) would be the natural third language given the cultural overlap with Korean and the demand from the Japanese tech and travel market.

13.3 Long-Term Vision

The long-term vision for VibeWeather is to become the reference example of what a beautiful, scientifically accurate, culturally aware weather application can be -- built with minimal dependencies and maximum attention to the user experience.

- **Push Notification System:** Browser Push API alerts for severe weather in saved cities, even when the app is not open.
- **Accessibility (WCAG 2.1 AA):** ARIA labels, full keyboard navigation, sufficient color contrast ratios for all status indicators.
- **AI-Powered Descriptions:** Use Claude API to generate a daily natural-language weather summary in the active language: "Today in Seoul feels cooler than it looks -- the wind is making 18 degrees feel like 14. Pack a light jacket."

- **React Native Mobile App:** A native mobile version that can access device sensors, deliver true push notifications, and provide home screen widgets.
- **Open Source Release:** Full public GitHub release with documentation, allowing developers to fork, extend, and localize for other languages and regions.

Appendix: Reference Tables & Code

A. WeatherAPI JSON Field Map

Complete mapping of WeatherAPI response fields to VibeWeather UI elements and processing:

JSON Path	JS Variable	UI Element	Processing Applied
location.name	location.name	#cityName	Concat with region/country
location.tz_id	tzId	Live clock	Passed to updateLiveClock()
location.localtime	localTimeStr	Background, clock hour	Split on space, parse hour int
current.temp_c	temp	#temp	Math.round() + "degC"
current.condition.text	conditionText	#description	Mapped to Korean in KO mode
current.condition.code	conditionCode	Icon + outfit + alerts	11-flag boolean cascade
current.is_day	isDay	timeIcon src	Ternary: sunny.png / moon.gif
current.humidity	humidity	#humidityVal	Dew point calc, comfort label
current.wind_kph	windSpeedKmh	#windVal	Round, real feel input, driving
current.uv	uvIndex	#uvVal	6-level label mapping
current.vis_km	visKm	#visVal	5-level label mapping
current.pressure_mb	pressureHpa	#pressureVal	6-level stability mapping
current.air_quality.us-epa-index	aqi	#aqiVal	6-level AQI mapping
forecastday[0].astro.sunrise	--	#sunriseTime	Direct display
forecastday[0].astro.sunset	--	#sunsetTime	Direct display
forecastday[0].astro.moon_phase	--	#moonPhaseText	8-entry dict lookup + emoji
forecastday[0].day.mintemp_c	min	#minTemp	Math.round() + "deg"
forecastday[0].day.maxtemp_c	max	#maxTemp + bar	Temp bar position calc
forecastday[0].day.daily_chance_of_rain	precipChance	#precipVal + bar	% display + bar width
forecastday[0].hour[]	allHours[0..23]	Hourly strip	Filter > localTime, slice 12
forecastday[1..5]	forecastDay[1..5]	5-day strip	renderForecast() iteration

B. Condition Code to Icon Map

Flag	Condition Codes	Day Icon	Night Icon
isThunder	1087, 1273-1282	Strom.gif	Strom.gif

isBlizzard	1117	blizzard.png	blizzard.png
isSnow	1210-1282 (no thunder)	snow.png	snow.png
isHail	1237, 1261, 1264	hail.gif	hail.gif
isSleet	1204-1207	sleet.png	sleet.png
isRain	1180-1201, 1063	rain day.png	rain.png
isDrizzle	1150-1171, 1072	rain day.png	rain.png
isFog	1135, 1147	fog.png	fog.png
isOvercast	1009	Cloudy_overcast.png	Cloudy_overcast.png
isCloudy	1003-1030	cloudy day.png	cloudy night.png
Clear (default)	1000	sunny.png	moon.gif

C. Key JavaScript Functions Reference

Function	Parameters	Returns	Purpose
getVibe(searchInput, originalQuery)	city string, original query	void	Main async function: fetches weather, calls renderWeather()
renderWeather(data, showDashboard)	API response object, bool	void	Updates all 35+ DOM elements from cached data
updateLiveClock(tzId, temp)	IANA timezone, temperature	interval ID	Starts 1s clock tick for searched city local time
updateDynamicBackground(hour)	Local hour integer (0-23)	void	Selects and loads time-appropriate background video
calcRealFeel(T, v, H)	temp, wind_kph, humidity	float	3-branch real feel: Wind Chill / Heat Index / Steadman
getWeatherIcon(code, text, isDay)	condition code, text, bool	object (flags + gif)	11-flag boolean cascade for icon selection
getHumidityVibe(dewPoint)	dew point in C	object {vibe, color}	Maps dew point to 9-level comfort label + color
getWindFeeling(rf)	real feel temperature	object {text, color}	Maps apparent temp to 11-level wind feeling label
getOutfit(code, text, T, rain, thunder, snow, fog)	multiple weather flags	object {outfit, emo, priority}	Priority-ordered outfit recommendation + animation
getNextUp(nextData, T, wind)	hour object, current temp+wind	object {vibe, emoji, 5-min}	5-min lookahead predictive Coming Up widget
getAlertMsg(flags, wind, uv, T)	condition flags + values	string or null	Generates alert banner text for extreme conditions
setLang(lang)	"ko" or "en"	void	Updates currentLang, refreshes all text, re-renders
t(ko, en)	Korean string, English string	string	Returns correct language string based on currentLang
playAmbient(name)	track name string	void	Switches ambient audio track smoothly
renderForecast(forecastDay)	forecast array	void	Builds 5-day forecast card HTML
renderHourly(hours, localTime)	hours array, localtime string	void	Builds 12-hour forecast card HTML
getMoonData(moonPhaseRaw)	moon phase string	object {emoji, label}	8-entry dictionary: moon phase string to emoji + label

D. Deployment Checklist

For deploying Version 2 (GitHub Pages) from scratch:

1. Create a GitHub repository (public for free GitHub Pages).
2. Upload all project files: index.html, script.js, style.css, assets/ folder.
3. Insert WeatherAPI.com key into the apiKey constant in script.js.
4. Enable GitHub Pages in repository Settings > Pages > Deploy from main branch.
5. Wait ~60 seconds for the first deployment to complete.
6. Visit <https://{username}.github.io/{repo-name}/> to verify.
7. Test Korean city search, language toggle, geolocation, share button.
8. Test on mobile browser (iOS Safari and Android Chrome).
9. Verify backdrop-filter glassmorphism renders (requires HTTPS -- GitHub Pages provides this).
10. Verify audio plays after user gesture (mute toggle first use).
11. Set a spending limit on WeatherAPI.com to prevent unexpected API charges.

E. CSS Custom Properties & Design Tokens

VibeWeather uses three CSS custom properties (design tokens) defined on :root. These are referenced throughout the stylesheet for consistency:

Variable	Value	Usage
--glass	rgba(255, 255, 255, 0.1)	Card background transparency (main card, vibe cards, extra container)
--text	#ffffff	Default text color for all body content
--neu-shadow	10px 10px 20px rgba(0,0,0,0.3), -5px -5px 15px rgba(255,255,255,0.05)	Dual neumorphic shadow applied to all card elements

F. Complete Glassmorphism Card Specification

Every card in the UI -- .main-card, .extra-container, and all .vibe-card elements -- shares this exact CSS block. The consistency is intentional: the entire visual language is unified through one reusable rule:

```
.main-card, .extra-container, .vibe-card {
background: rgba(0, 0, 0, 0.35);
backdrop-filter: blur(8px);
-webkit-backdrop-filter: blur(8px); /* Safari */
border-radius: 30px;
box-shadow: var(--neu-shadow);
border: 1px solid rgba(255, 255, 255, 0.1);
box-sizing: border-box;
text-align: center;
}
```

The -webkit-backdrop-filter rule is required for Safari compatibility. Without it, Safari renders the cards without the blur effect. The standard backdrop-filter alone is insufficient for cross-browser support.

G. All 6 CSS Keyframe Animation Definitions

Each animation is paired with a utility class. JavaScript assigns the class to the appropriate emoji or icon element dynamically:

shiverVibe / .motion-shiver -- Cold/Snow (0.15s infinite)

```
@keyframes shiverVibe {
0% { transform: translate(1px,1px) rotate(0.5deg); }
25% { transform: translate(-1px,-1px) rotate(-0.5deg); }
50% { transform: translate(-1px,1px) rotate(0.5deg); }
75% { transform: translate(1px,-1px) rotate(-0.5deg); }
100% { transform: translate(1px,1px) rotate(0.5deg); }
}
```

bounceVibe / .motion-bounce -- Hot/Sunny (0.8s infinite)

```
@keyframes bounceVibe {  
  0%, 100% { transform: translateY(0); }  
  50% { transform: translateY(-20px) scale(1.1); }  
}
```

floatVibe / .motion-float -- Default/Mild (3s infinite)

```
@keyframes floatVibe {  
  0%, 100% { transform: translateY(0) rotate(0deg); }  
  50% { transform: translateY(-12px) rotate(3deg); }  
}
```

swayVibe / .motion-sway -- Windy (2s infinite)

```
@keyframes swayVibe {  
  0%, 100% { transform: rotate(-10deg); transform-origin: bottom center; }  
  50% { transform: rotate(10deg); transform-origin: bottom center; }  
}
```

rainDrop / .motion-rain -- Rain (1.5s infinite)

```
@keyframes rainDrop {  
  0% { transform: translateY(-5px); opacity: 0.8; }  
  50% { transform: translateY(5px); opacity: 1.0; }  
  100% { transform: translateY(-5px); opacity: 0.8; }  
}
```

stormShake / .motion-storm -- Thunder (0.5s infinite)

```
@keyframes stormShake {  
  0%, 100% { transform: translateX(0); }  
  10%, 30%, 50%, 70%, 90% { transform: translateX(-4px); }  
  20%, 40%, 60%, 80% { transform: translateX(4px); }  
}
```

H. Real Feel -- Worked Calculation Examples

Three worked examples show how the real feel temperature is computed across all three branches:

Example 1: Seoul Winter Morning -- Wind Chill Branch

```
Input: T = 2 degC, V = 25 km/h, H = 55%
Branch selected: Wind Chill (T <= 10 AND V >= 4.8)
WC = 13.12 + (0.6215 x 2) - (11.37 x 25^0.16) + (0.3965 x 2 x 25^0.16)
= 13.12 + 1.243 - (11.37 x 1.783) + (0.793 x 1.783)
= 13.12 + 1.243 - 20.27 + 1.414
= -4.49 degC
Humidity adjustment: H=55 > 30, no adjustment applied.
Result: Real Feel = -4.5 degC | Wind label: "Freezing"
```

Example 2: Bangkok Summer Afternoon -- Heat Index Branch

```
Input: T = 34 degC, V = 8 km/h, H = 75%
Branch selected: Heat Index (T >= 27 AND H >= 40)
HI = -8.784 + (1.611x34) + (2.339x75) + (-0.146x34x75) + (-0.012x34^2)
+ (-0.016x75^2) + (0.002x34^2x75) + (0.001x34x75^2) + (-3.8e-6x34^2x75^2)
= -8.784 + 54.77 + 175.43 - 372.3 - 13.87 - 90.0 + 174.0 + 191.25 - 124.7
~= 42.1 degC
Wind adjustment: V=8 < 15, no adjustment applied.
Result: Real Feel = 42.1 degC | Wind label: "Extreme Heat"
```

Example 3: London Autumn Afternoon -- Steadman Branch

```
Input: T = 17 degC, V = 15 km/h (4.17 m/s), H = 65%
Branch selected: Steadman (not cold+windy, not hot+humid)
e = (65/100) x 6.105 x exp((17.27 x 17) / (237.7 + 17))
= 0.65 x 6.105 x exp(1.155) = 0.65 x 6.105 x 3.174 = 12.60 hPa
AT = 17 + (0.33 x 12.60) - (0.70 x 4.17) - 4.00
= 17 + 4.16 - 2.92 - 4.00 = 14.24 degC
Mild+humidity adjustment: T=17 in range, H=65>40 but <70, no adjustment.
Result: Real Feel = 14.2 degC | Wind label: "Chilly"
```

I. Korean City Coordinate Dictionary -- Sample Entries

A representative sample from the 150+ entry coordinate dictionary. Entries cover Seoul districts, Gyeonggi-do satellite cities, and all provincial capitals:

Korean Name	English Name	Latitude	Longitude	Category
서울	Seoul	37.5665	126.9780	Special City
강남구	Gangnam-gu	37.5172	127.0473	Seoul District
마포구	Mapo-gu	37.5638	126.9084	Seoul District

■■■	Songpa-gu	37.5145	127.1059	Seoul District
■■	Incheon	37.4563	126.7052	Metropolitan City
■■	Suwon	37.2636	127.0286	Gyeonggi-do
■■	Seongnam	37.4449	127.1388	Gyeonggi-do
■■	Goyang	37.6583	126.8320	Gyeonggi-do
■■	Busan	35.1796	129.0756	Metropolitan City
■■■■	Haeundae-gu	35.1628	129.1636	Busan District
■■	Daegu	35.8714	128.6014	Metropolitan City
■■■■■	Gwangju	35.1595	126.8526	Metropolitan City
■■	Daejeon	36.3504	127.3845	Metropolitan City
■■	Ulsan	35.5384	129.3114	Metropolitan City
■■	Sejong	36.4800	127.2890	Special City
■■	Jeonju	35.8242	127.1479	North Jeolla
■■	Gangneung	37.7519	128.8761	Gangwon-do
■■	Gyeongju	35.8562	129.2247	North Gyeongsang
■■	Jeju	33.4996	126.5312	Jeju Island
■■■	Seogwipo	33.2541	126.5600	Jeju Island

Input normalization strips common suffixes before lookup. For example, "■■■" and "■■■■■" both resolve to the "■■■" entry. The stripping regex: `/(■■■|■■■|■■■■■|■■■■■■|■■■■■■|■■■■■■|■■|■■)$$/`

J. Share Weather -- Output Format

When the user taps Share Weather, the app generates a pre-formatted text summary in the active language. The format differs between Korean and English modes:

English Mode Output

```
Weather in Seoul, South Korea
Temperature: 24 degC -- Sunny
Feels like: 19.2 degC
Humidity: 21%
Wind: 13 km/h
Checked on VibWeather Pro
```

Korean Mode Output

Seoul, South Korea 🇰🇷
🌡️: 24 degC -- 🌡️
🌧️: 19.2 degC
💧: 21%
🌬️: 13 km/h
VibWeather Pro 🌤️ 🌤️ 🌤️

On browsers supporting the Web Share API (Chrome, Safari, Edge), the native share sheet opens -- allowing the user to share directly to messaging apps, social media, email, or AirDrop. On Firefox, the text is copied to clipboard and a confirmation message is briefly shown on the button.

K. Browser Feature Compatibility Reference

VibeWeather uses several modern Web APIs. This table summarizes browser support for each feature and the fallback behavior where applicable:

Feature / API	Chrome	Firefox	Safari	Edge	Fallback Behavior
CSS backdrop-filter (Glassmorphism)	76+	103+	9+ (-webkit-)	79+	Cards render with transparent BG but no blur -- still readable
HTML5 Video autoplay (muted)	All	All	All (playsinline)	All	No video = dark gradient BG only, app fully functional
Web Share API	89+	Partial	12.1+	93+	Falls back to navigator.clipboard.writeText()
navigator.geolocation	All	All	All	All	Graceful error message if permission denied
localStorage	All	All	All	All	App works without it; no history chip shown
CSS Grid	All	All	All	All	No fallback needed -- universal support
CSS Custom Properties (--variables)	All	All	All	All	No fallback needed -- universal support
fetch() async/await	All	All	All	All	No fallback needed -- universal support
Audio autoplay	Blocked*	Blocked*	Blocked*	Blocked*	Muted by default; mute toggle provides user gesture
Intl.DateTimeFormat (timezone)	All	All	All	All	No fallback needed -- universal support
CSS scroll-behavior	All	All	All	All	No fallback needed -- universal support

* Audio autoplay is blocked by design in all modern browsers without a prior user gesture. The muted-by-default approach is the correct solution, not a workaround.

L. Project Development Timeline

VibeWeather was developed over approximately 11 months across two distinct phases. The timeline below reflects the key milestones:

Period	Phase	Key Milestones
Jun 2024	V1: Foundation	Project concept. First HTML structure. Basic city search with WeatherAPI.
Jul 2024	V1: Core Features	Main weather card complete. Temperature bar. CSS glassmorphism design.
Aug 2024	V1: Lifestyle Cards	All 12 vibe grid cards implemented. Real feel 3-branch model added.
Sep 2024	V1: Forecast & Polish	5-day and hourly forecast rendering. Background video engine. PHP proxy.
Oct 2024	V1: Extras	Moon phase. Solar animations. Share button. Alert banner. localStorage.
Nov 2024	V1: Complete	Testing across browsers. Performance tuning. Version 1 deployed.
Dec 2024	V2: Planning	Korean language requirement identified. Architecture redesign for bilingual.
Jan 2025	V2: KO Dictionary	150+ Korean city coordinate dictionary built. Input detection logic.

Feb 2025	V2: t() Engine	Bilingual t() helper implemented. All label generators made bilingual.
Mar 2025	V2: Toggle UI	KO/EN toggle button. setLang() function. Data attribute refactor.
Apr 2025	V2: Audio Engine	Full ambient audio system activated. 4 tracks matched to conditions.
May 2025	V2: Complete	Version 2 deployed on GitHub Pages. Full mobile testing. Documentation.

M. Tools, Libraries & Resources Used

VibeWeather was built with deliberate minimalism -- zero runtime JavaScript dependencies. The following external resources were used:

Resource	Type	Version / Tier	Purpose
WeatherAPI.com	REST API	Free tier (1M calls/month)	All weather data: current, forecast, AQI, astronomy
Nominatim (OpenStreetMap)	Geocoding API	Free, rate-limited	Korean city name to coordinate fallback
Google Fonts -- Poppins	Font CDN	300 / 400 / 600 weights	All UI typography
Font Awesome (icons)	Icon library	v6 CDN	Location pin icon in city name display
GitHub Pages	Static hosting	Free tier	Production deployment of Version 2
XAMPP	Local server	8.2+	Local development for Version 1 (PHP)
VS Code	Code editor	Latest	Primary development environment
Chrome DevTools	Browser tools	Built-in	Debugging, responsive testing, network analysis
Canva / Video editors	Asset creation	Free tier	Background video assets and audio preparation
Cursor (AI-assisted)	IDE / AI tool	--	Vibe coding -- rapid UI iteration with AI assistance
Claude AI	AI assistant	--	Architecture discussions, code review, documentation

The zero-dependency philosophy means no npm, no bundler, no build step. The entire application is served as plain HTML/CSS/JS files. This reduces deployment complexity to a single file upload and makes the codebase accessible to any developer without a setup process.

N. Moon Phase Dictionary & Astronomy Data

VibeWeather maps the moon_phase string from WeatherAPI's astronomy endpoint to an emoji and bilingual label using a hardcoded 8-entry dictionary. The full lunar cycle is covered:

moon_phase String	Emoji	Korean Label	English Label	Description
New Moon	🌑	☾	New Moon	Moon not visible -- darkest night
Waxing Crescent	🌒	☾☾	Waxing Crescent	Thin crescent on right side
First Quarter	🌓	☾☾☾	First Quarter	Right half illuminated
Waxing Gibbous	🌔	☾☾☾☾☾	Waxing Gibbous	More than half, growing
Full Moon	🌕	☾☾☾	Full Moon	Fully illuminated -- brightest night

Waning Gibbous	■	■■■■ ■	Waning Gibbous	More than half, shrinking
Last Quarter	■	■■■	Last Quarter	Left half illuminated
Waning Crescent	■	■■■	Waning Crescent	Thin crescent on left side

Beyond moon phase, the astronomy endpoint also provides sunrise and sunset times which power the animated solar row. The sunrise element uses a CSS keyframe animation (animate-sunrise) that moves a golden radial-gradient div from lower-left to upper-center, simulating an arc across the sky. The sunset uses a complementary drop animation.

The moon emoji element (.moon-vibe-animation) has its own CSS animation: a gentle 5-second float with a 5-degree rotation and a pulsing drop-shadow glow. This gives the moon a sense of softly drifting through the night sky rather than sitting static on the screen.

O. Security Considerations Summary

VibeWeather approaches security pragmatically, acknowledging the difference between a personal portfolio project and a commercial production application:

Concern	Version 1 Approach	Version 2 Approach	Production Best Practice
API key exposure	PHP proxy -- key never in browser	Key in JS source (GitHub Pages build)	Server-side function (Netlify/Vercel Edge)
Input sanitization	PHP urlencode() on city name	JS encodeURIComponent() before fetch	Server-side validation + rate limiting
HTTPS requirement	Depends on hosting config	GitHub Pages always HTTPS	Always enforce HTTPS in production
Rate limiting	API has built-in limits	API has built-in limits	Server-side cache + rate limiter
CORS	Same-origin (PHP proxy)	Direct API call (no CORS issue)	Proxy pattern eliminates CORS concerns
XSS prevention	innerHTML used cautiously	innerHTML used cautiously	Use.textContent for user-supplied data
Data storage	localStorage (client-only)	localStorage (client-only)	No sensitive data in localStorage

Project Summary & Key Takeaways

VibeWeather is more than a weather application. It is a demonstration of what thoughtful, user-centered engineering looks like when the developer is also the user researcher, the designer, the backend engineer, and the frontend developer -- all driven by a genuine desire to solve a real problem.

What was built

A full-stack weather intelligence application in two editions: an English edition with PHP backend proxy, and a bilingual Korean-English edition deployed on GitHub Pages. Both editions implement scientific temperature modeling, 12 lifestyle intelligence cards, dynamic time-of-day video backgrounds, and ambient audio.

What makes it different

Three scientific real feel models instead of one API value. Dew point comfort scale instead of raw humidity. 150+ Korean city coordinate system for bilingual support. Living video background keyed to the searched city's local time. Zero frontend frameworks. Complete real-time KO/EN language toggle with computed value re-rendering.

Technical skills demonstrated

Vanilla JavaScript state management and data flow design. REST API integration and async/await patterns. PHP proxy security pattern. CSS Grid and Flexbox responsive layout. HTML5 Video and Audio APIs. localStorage and Web Share API. CSS keyframe animation library. Bilingual string management architecture. OpenStreetMap Nominatim geocoding.

Design skills demonstrated

Glassmorphism design system implementation. Animation-as-communication (weather-responsive emoji motion). Color-coding as a consistent semantic language. Information density without visual clutter. Mobile-first responsive design tested on real devices.

The meta-skill

Shipping Version 1, learning from it, and iterating to Version 2 with a significantly more complex feature set -- while maintaining code quality and architectural integrity. This incremental delivery approach is the most transferable skill demonstrated in this entire project.

13

Chapters of documentation

41

Pages of technical detail

2

Production deployments

11

Months of development

"A good weather app tells you the temperature. A great weather app tells you how to feel about it."